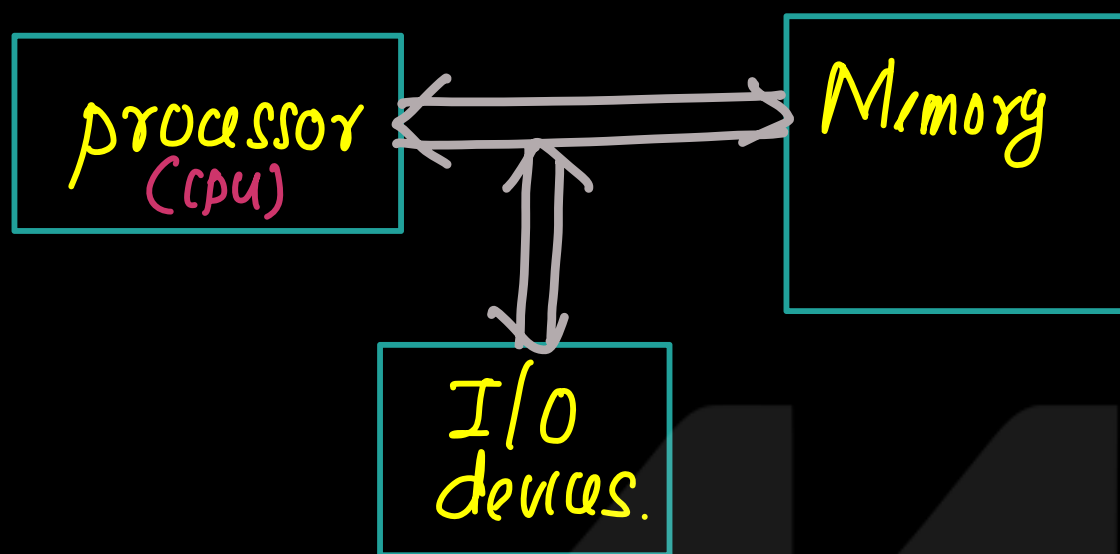


Basic Structure of Computer (Note)

(N)

→ there are 4 main structural components

- ① Central processing units (CPU) / processor
- ② Memory
- ③ I/O devices
- ④ System Bus.



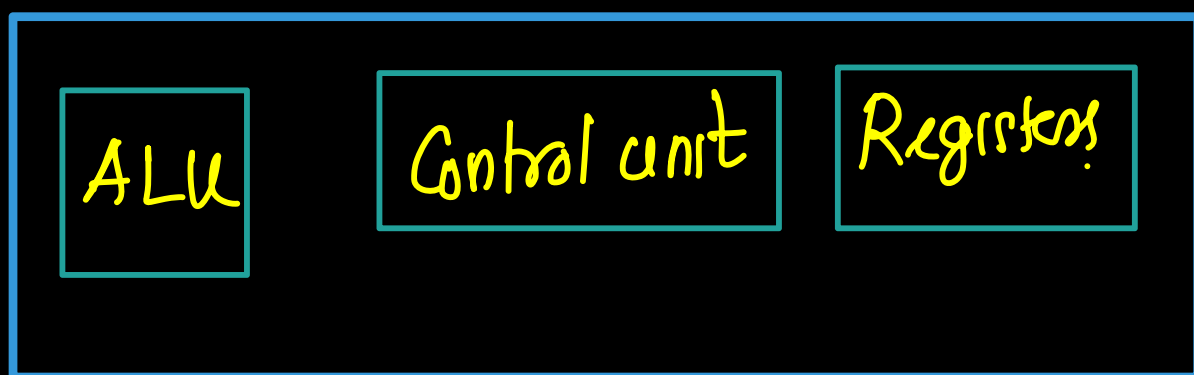
→ Central Processing Unit (CPU)

→ it is the brain of the computer and is responsible for the execution of an instruction

→ it controls the operation of computer and performs its data processing function

→ CPU has 3 main parts

- ① ALU, ② Control unit ③ Registers



① ALU : Performs all arithmetic and logical operation

② Control unit : Control unit decode the instruction and generate control signal to coordinall all activities.

③ Registers : these are the high speed storage location inside the CPU used to store data / instruction

eg: Program Counter
Accumulator
instruction register

② Memory:

- Memory is used to store the information (data & instruction)
- memory is divided into small units called memory location. each memory location has a unique number assigned to it, called memory address.

③ I/O devices

→ these are the devices used to move data b/w the computer and its external environment

— input devices:

these devices allow the user to enter data and instruction into the computer

eg: keyboard

— output devices

these devices present processed data to the user

eg: monitor

System bus:

— These mechanisms provide communication b/w CPU, memory and I/O devices.

— these are the set of lines used to transfer information

— There are 3 types of buses

① Address Bus — it carries address of memory location

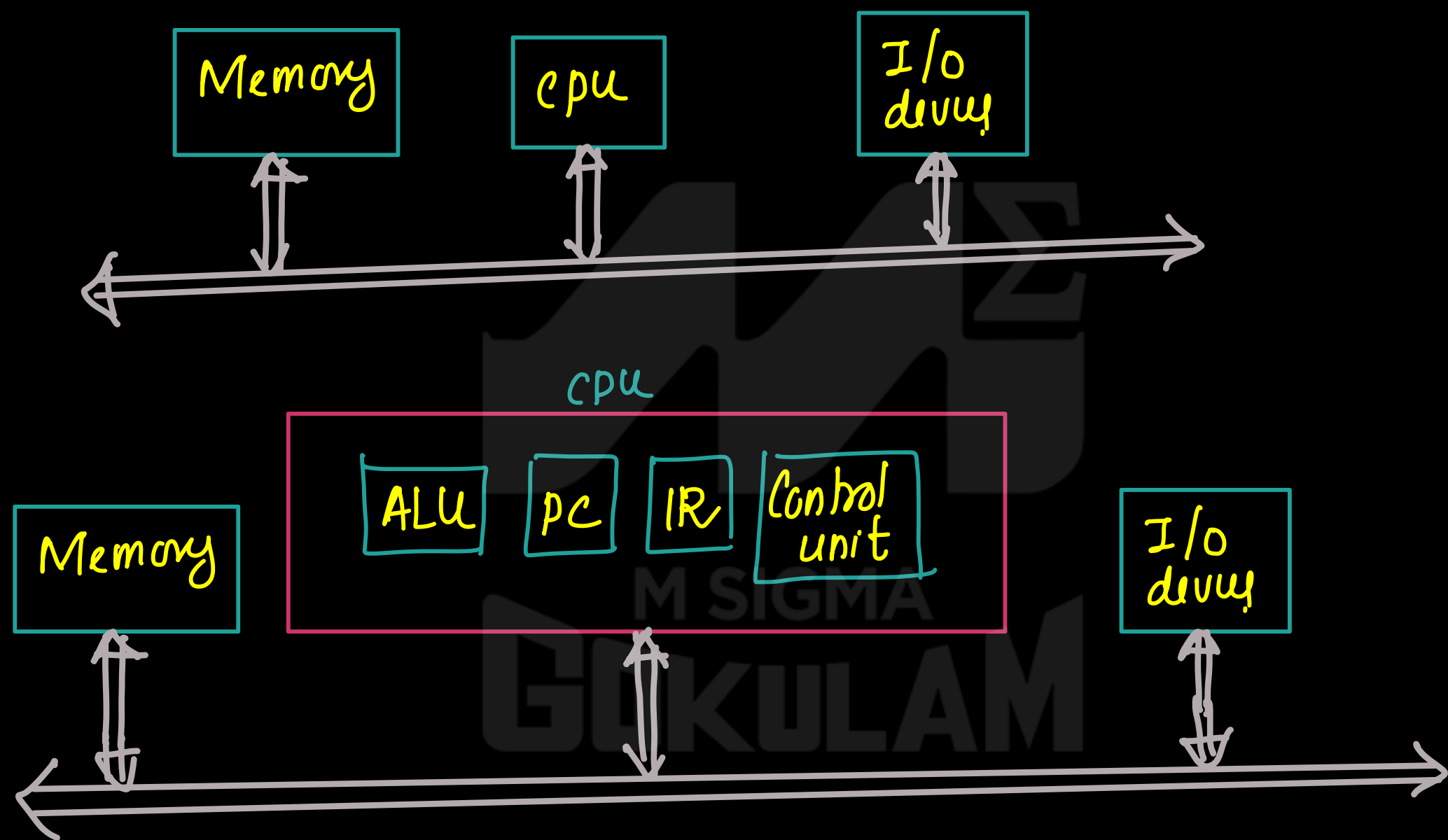
② Data Bus — it carries data to and from the processor

③ Control Bus — it carries control signals (Read signal, write signal)

Basic Operational Concept (Note)

(N)

- Basic operation concepts describes how a computer perform a task.
- CPU execute a task through the following steps.
 - ① Fetch
 - ② Decode
 - ③ execute
 - ④ store.
- Consider the example of Adding two numbers. Instruction for the task is stored in the memory location



Fetch

- ① Program Counter is a register inside the CPU. This register contains address of the next instruction to be executed.
- ② CPU fetches the next instruction from the memory. The program counter (PC) gives the address of the instruction. Instruction is brought into the instruction register present inside the CPU.

Decode

- Control units read the instruction in the IR and decode the instruction
- Control unit identify the operation to perform and send control signal to the different parts of CPU to perform the operation

Execute.

- CPU execute perform the operation specified by the instruction
- Result of execution is stored in a register or main memory for later use

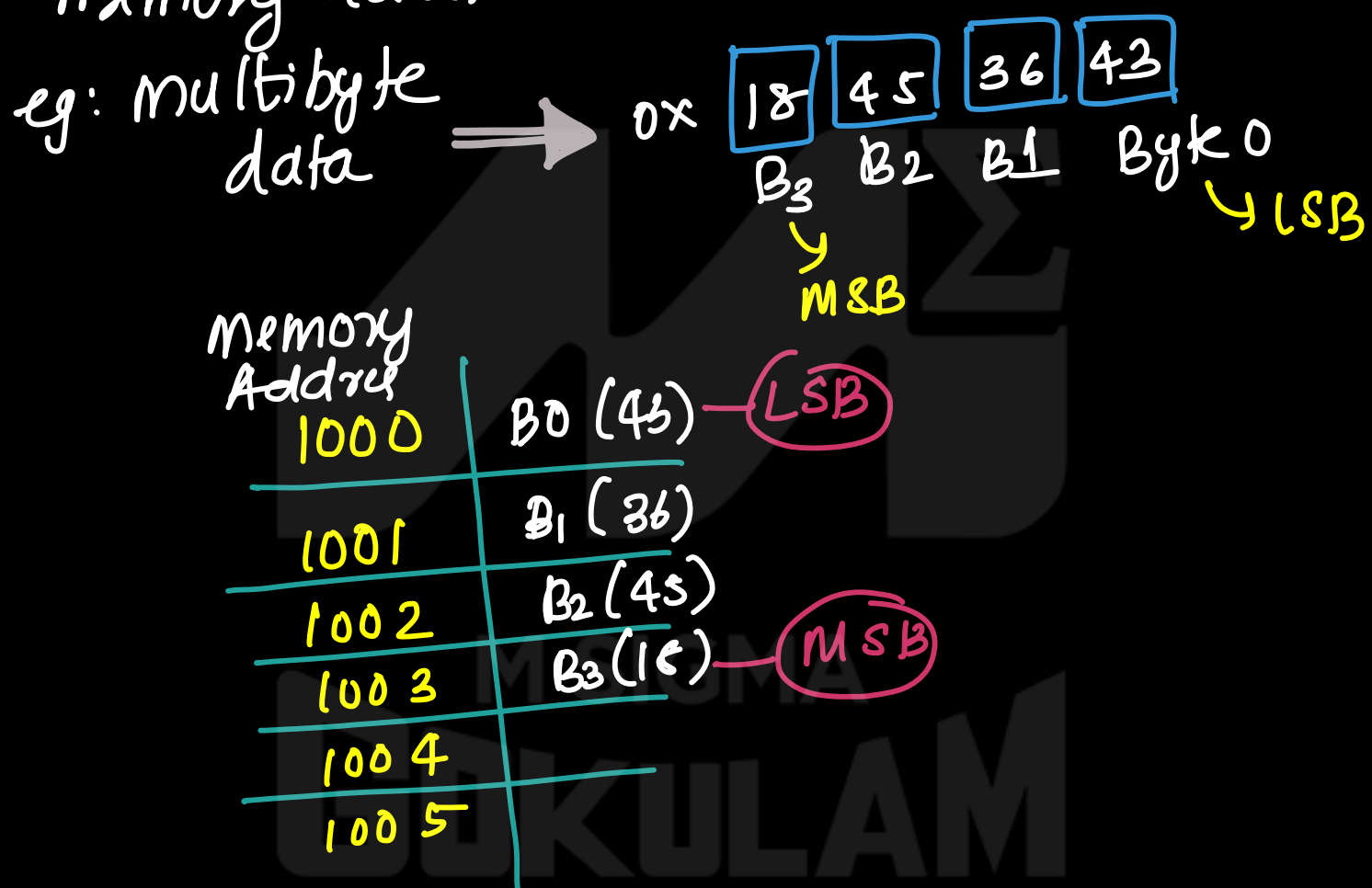


Endianness (Note)

- Endianness specifies the order in which a multi-byte data is stored in the memory.
- Endianness refers to the byte ordering method used to store multi-byte data in memory.

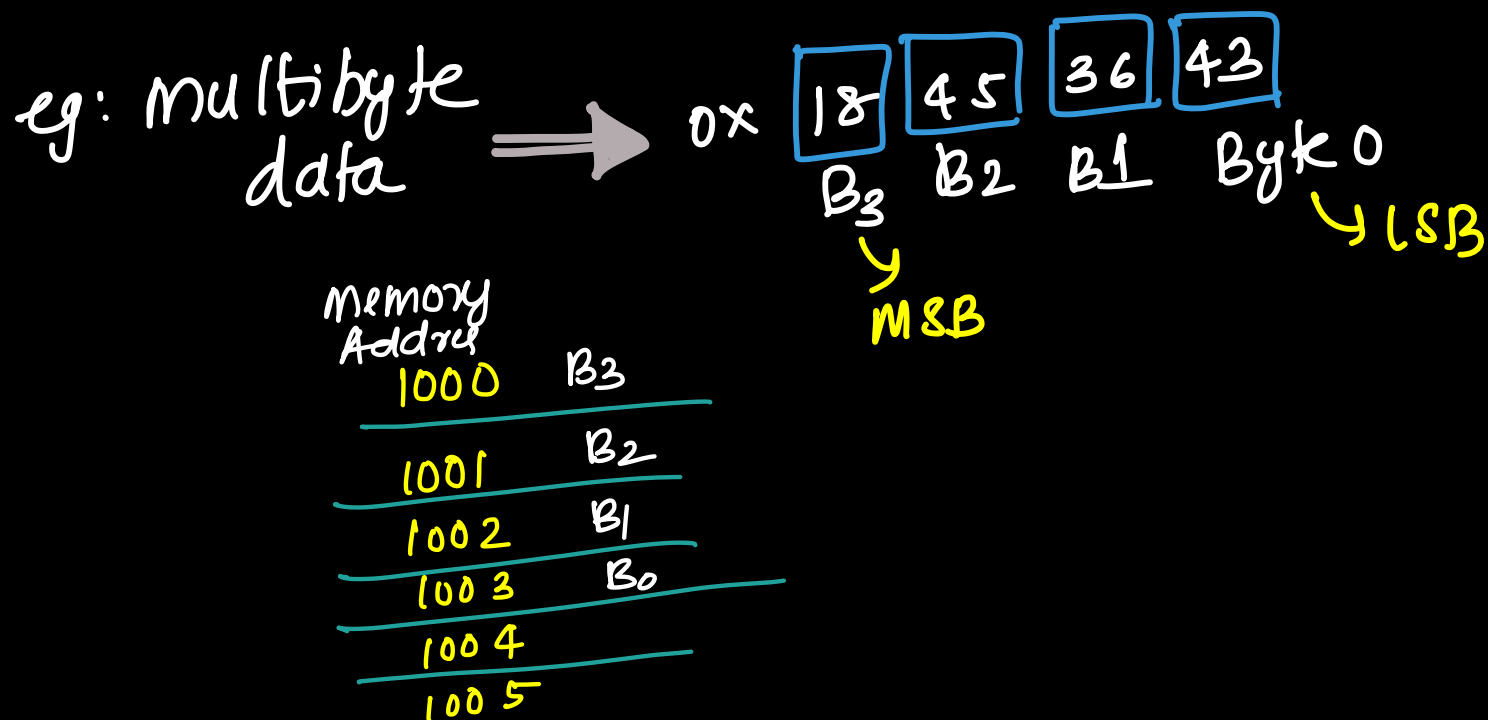
Types of endianness.

- (i) Little endianness
- Least significant byte (LSB) is stored at lowest memory location.



(2) Big endian

The most significant byte (MSB) stored at lowest memory location



Module - 1



A 32-bit integer 0x12345678 is stored at memory location 1000. If the system is Little-Endian, what is the byte content of location

Little endian → LSB stored at lower memory location

big endian - LSB at higher memory location

Multi byte data = 0x 12 34 56 78

↓
MSB

LSB → lower Memory Location

Memory location	Data
1000	LSB 78
1001	56
1002	34
1003	MSB 12

LSB.

Big endian

LSB stored at higher memory location

1000	MSB 12
1001	34
1002	56
1003	LSB 78
1004	

Assembly language (Note)

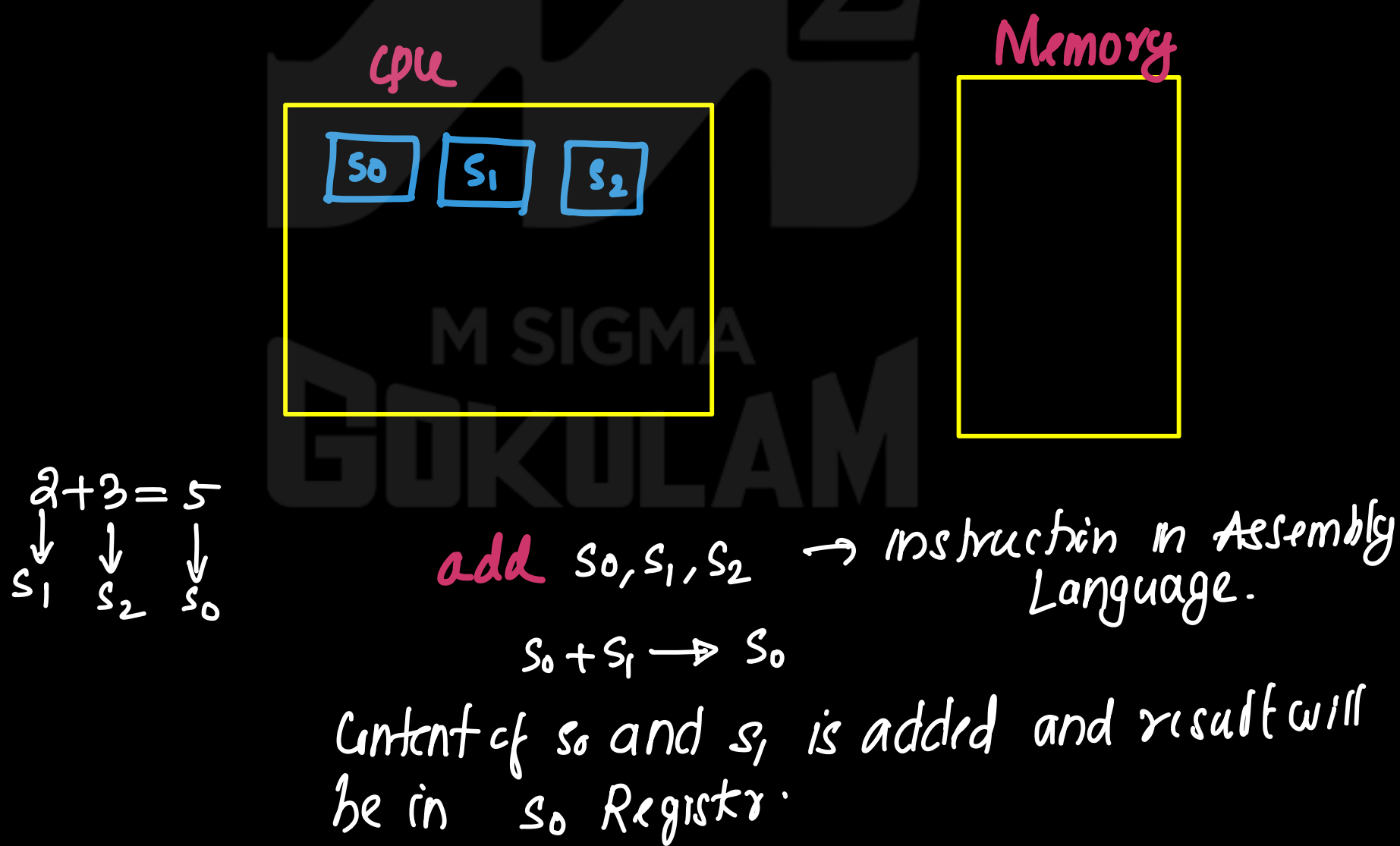
(N)

→ Assembly language is the human readable representation of the computer native language (Machine language)

1010110 — Computer readable format of instructions
ADD S₀, S₁, S₂ — Human readable format of instruction

→ Each assembly language instruction specifies both the operation to perform and operands on which operates.

eg: We had to add two numbers stored at two registers S₁, S₂ and store the result in a Register S₀



2 + 3 = 5
↓ ↓ ↓
S₁ S₂ S₀

`add S0, S1, S2` → instruction in Assembly Language.

$S_0 + S_1 \rightarrow S_0$

Content of S₀ and S₁ is added and result will be in S₀ Register.

`add S0, S1, S2`
↓
mnemonics operands

→ Each assembly language instruction specifies an operation to be performed (mnemonics) and the data on which the operation acts. (operands)

Mnemonics:

- it represent actual operation
- it is very easy to remember

eg: ADD — Addition
SUB — subtraction
LW — Load word
SW — store word
MOV — move data.

Operands:

- operands specify the data or location where the data is present. Data may be present register inside the CPU. or Data may be present inside the memory location

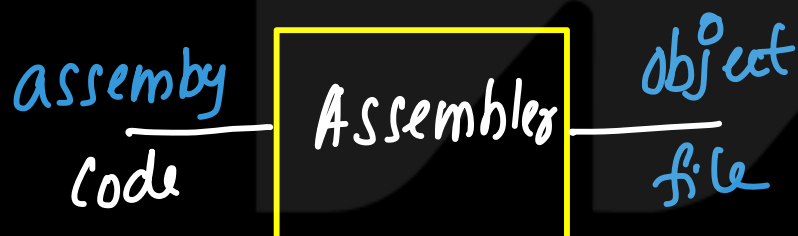


Assembler and Linker (Note)

- Assembly language cannot be executed by the CPU because it uses human readable mnemonics and symbolic address.
- CPU can understand only machine code (binary instructions).
- To execute an assembly program, it must be converted to an executable file, which is done in two main steps.
 - ① Assembling and Linking.

Step 1 Assembler

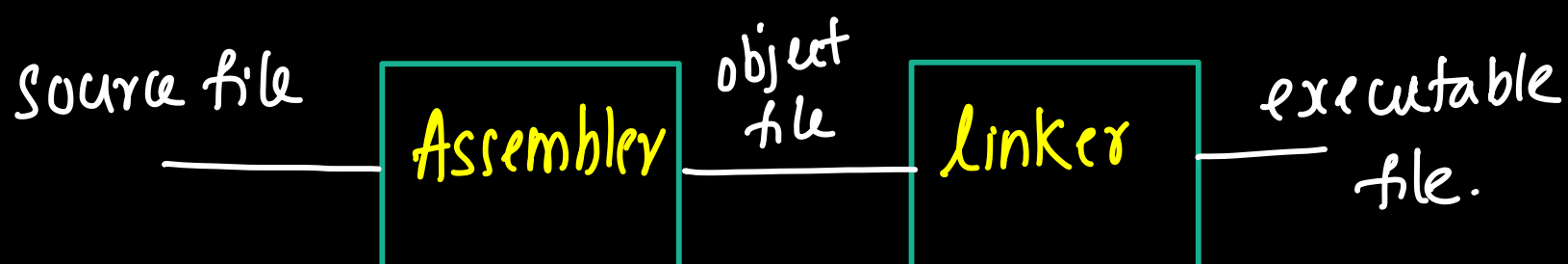
Assembler is a system program that converts assembly language program into object file containing machine code.



Step-2: Linker

- The object file generated by the assembler is not an executable file.
- Linker converts this object file to an executable file.
- Linker is a system program that produces an executable file that the CPU can run.

After linking, the program is loaded into the memory and executed by the CPU.



Assembler directives (Note)

(N)

Assembly directives are the special instruction in an assembly program that tell the assembler how to process the program. But assembly directives are not executed by CPU.

Commonly used directives

① .data ② .text ③ .word ④ .byte, ⑤ .end

① .data → This instruction marks the start of a data segment where the variables are defined and initialized.

② .text → Indicates the beginning of a section containing executable instructions.

③ .word — Used to allocate and initialize memory storage for data of specific size.
 .byte — allocate 1 byte memory space
 .word — allocate 4 byte memory space
 .byte — allocate 1 byte memory space

④ .end — Mark the end of the source code file

.data

num1 = .word 2

num2 = .word 5

.text

lw s1, num1

lw s2, num2

add s0, s1, s2

.end

Assembly language and Assembler , linker and assembler Directives



Define Assembly Language. Explain its Advantages over Machine Language

Advantages of Assembly Language over Machine Language

Human Readability

Ease of Debugging

Writing assembly language programs is faster and less error-prone than writing machine language programs.

Compare Assembly Language and Machine Language

Assembly Language	Machine Language
Uses mnemonics and symbols	Uses binary codes (0s and 1s)
Easy to read and write	Very difficult for humans
Easier debugging	Debugging is difficult
Requires assembler for execution	Directly executed by CPU
Less error-prone	Highly error-prone

What is an assembler? Explain with a neat diagram how assembly language is converted into machine code.

Explain Why Assembly Language Cannot Be Directly Executed by CPU

- Assembly language uses mnemonics and symbolic names (like ADD, x1, num1)
- CPU understands only machine code (binary 0s and 1s)
- Role of Assembler: Converts assembly instructions into machine code
- Role of Object File: Stores the machine code in a structured format, along with data and symbol table
- Role of Linker: Resolves addresses and combines multiple object files into a single executable

Conclusion: Without assembling and linking, the CPU cannot directly execute assembly programs.

Describe the Steps Involved in Converting Assembly Language to Executable File

- Step 1: Assembly
- Step 2: Linking

State the significance of assembly directives with examples.

Explain with suitable examples commonly used assembler directives.

Differentiate between Instructions and Assembler Directives

Aspect	Instructions	Assembler Directives
Definition	Commands given to the CPU to perform specific operations.	Commands given to the assembler to organize the program or data.
Executed by	CPU executes instructions.	Not executed by CPU; interpreted by assembler.
Purpose	Perform arithmetic, logic, data movement, control operations.	Reserve memory, define data, control program assembly process.
Examples	ADD x1, x2, x3, LW x1, num1, SW x3, result	.data, .text, .word, .ascii
Effect on Program	Directly affects the behavior of the program at runtime.	Helps assembler prepare programs for execution; does not affect runtime directly.

CISC and RISC architecture (Note)

CISC (Complex instruction set computer)

RISC (Reduced instruction set computer)

- CISC and RISC are two fundamental approaches to processor design

- CISC Architecture

- This architecture provides a large set of instructions, including complex instruction that can perform multiple operations in a single instruction

- Single instruction can perform multiple low level operations

eg: add AX, [2000H]

operation. - ① load data from the memory
② perform addition
③ store the result.

- Features

① Single instruction can perform multiple low level operation

② Designed to reduce the no. of instructions per program. This reduces the program length.

③ Instruction length is different for different instruction (variable length instruction)

④ arithmetical and logical instruction can access memory. So only few registers are available inside the CPU

④ Execution time is different for different instructions (some instructions require one clock cycle, some require more than one clock cycle)

Disadvantage

- longer execution time per instruction
- Harder to implement pipelining.

eg: intel x86 family



RISC (Reduced instruction sets computer) (Note)

this architecture uses highly optimized instruction each of that can typically execute in one clock cycle

key feature:

- ① Each instruction perform a small, basic task
- ② All instruction are usually 32 bit (fixed length instructions).
- ③ arithmetic and logical instruction cannot access memory. Arithmetic and logical operations are done on registers
- ④ only load and store instruction can access the memory.
- ⑤ Large number of general purpose register are present in the RISC CPU
- ⑥ pipeline can be efficiently implemented.

eg: MIPS processor

Disadvantage

- . program may require more instructions for complex operation
- . may increase code size.

Instructions

Note

① Logical instruction
instruction to perform Logical operations like OR, AND, XOR
instructions 'and', 'or', 'xor'

① and

eg: and s1, s2, s3
s1 ← s2 and s3.

② or

or s1, s2, s3

③ xor

eg: xor s1, s2, s3

Source registers				
s1	0100 0110	1010 0001	1111 0001	1011 0111
s2	1111 1111	1111 1111	0000 0000	0000 0000
Assembly code		Result		
and s3, s1, s2	s3	0100 0110	1010 0001	0000 0000
or s4, s1, s2	s4	1111 1111	1111 1111	1111 0001
xor s5, s1, s2	s5	1011 1001	0101 1110	1011 0111

shift instructions: (Note)

— these instructions are used to shift the content of registers to left or write.

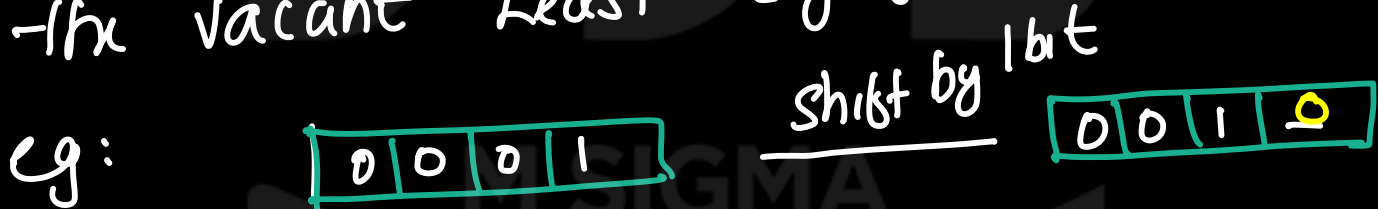


→ there are '3' shift instructions

- ① shift left Logical (sll)
- ② shift right logical (srl)
- ③ shift right arithmetic (sra)

① Shift Left Logical (sll)

Shift the content of register towards left and fill the vacant least significant bits with zeros.



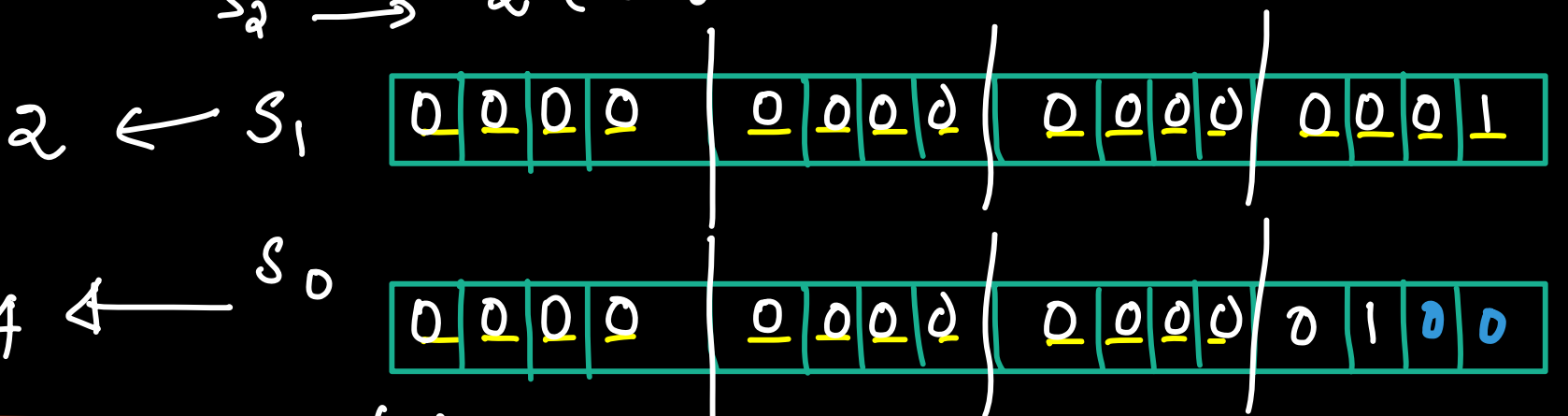
instruction 'sell'

eg: $s_{12} \quad s_0, s_1, s_2$

$S_1 \rightarrow$ Register contains the data
 $S_2 \rightarrow$ shift amount specified in S_2
 $S_0 \rightarrow$ shifted result will be present in the S_0

eg: $S_1 \rightarrow 0000 \quad 0000 \quad 0000 \quad 0000$

$S_1 \rightarrow 0000 \quad 0000 \quad 0000$
 $S_2 \rightarrow 2$ (shift left to 2 bit)



Shift left by 1 bit means multiplication by 2

(N)

Shift right Logical (srl)
Shift the content of register towards right and fill the vacant most significant bits with zeros.

0	0	0	1
---	---	---	---

instruction 'srl'

eg: s_0, s_1, s_2

$S_1 \rightarrow$ Register contains fix data
of amount specified in

$S_2 \rightarrow$ shift amount specified in S_2

$S_0 \rightarrow$ shifted result will be present in the S_0

eg: $S_1 \rightarrow$ 0000 0000 0000 0000

$S_2 \rightarrow 2$ (shift right to 2 bit)

S_1 0 0 0 0 | 0 0 0 0 | 0 0 0 0 | 0 0 0 1

S_0

<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>		<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>		<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>		0	0	0	0		0	1
----------	----------	----------	----------	--	----------	----------	----------	----------	--	----------	----------	----------	----------	--	---	---	---	---	--	---	---

right shift by one bit means divide no: by 2

③ shift right arithmetic

Shift right arithmetic
shift the content of a register to right
by preserving the sign of the number

cf :

1	0	0	1
---	---	---	---

gra by a hit

1	-	10	01
---	---	----	----

fill with MSB

Instruction

Sya s_1, s_2, s_3

1	1	1	0
---	---	---	---

Let the content of register S5 in a RISC V machine be - 1111 1111 0001 1100 0001 0000 1110 0111 - show the content of register t1 and t2 after executing the following instructions.

srls s1, s5, 17

srai t2, s5, 3

S₅ - 1111 1111 0001 1100 0001 0000 1110 0111

srls → shift right logically (fill MSB with 0).

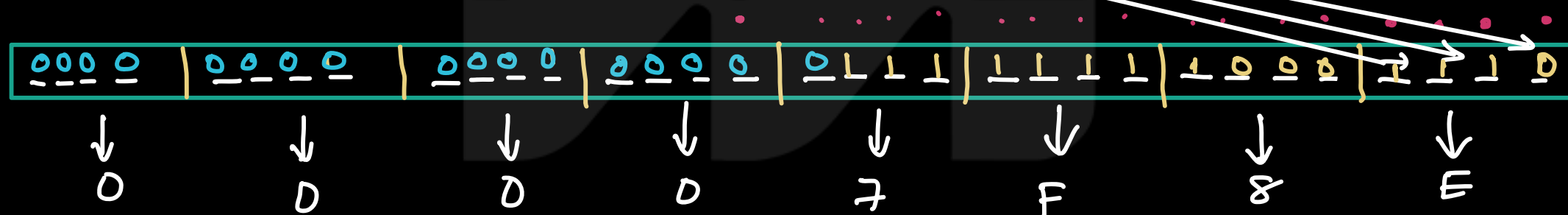
srls s1, s5, 17

shift the content of s5 right (17 bit) store result in s1

Before shift



After shift



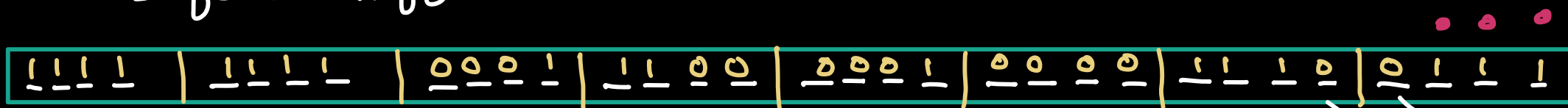
Content of s₁ = 0000 7F 8E

srai → shift right arithmetically (shift to right fill with MSB bits in vacant position)

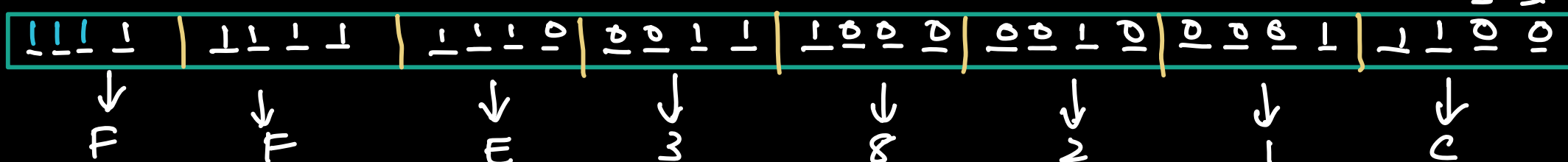
srai t2, s5, 3

shift the content of s5 (Right by 3 unit). and store the result in t2

Before shift



After shift



Branching (Note)

→ Under normal conditions processor execute instructions sequentially. i.e processor execute instructions one after another in a sequence

→

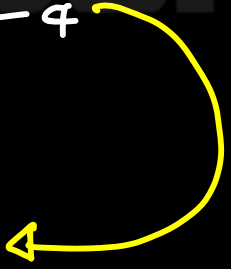
Memory Add	Stored instruction
0000 0000	Instruction-1
0000 0004	" 2
" 0008	" 3
" 000C	" 4
⋮	⋮
0050	
0054	
0058	

→ processor execute Instn 1, then
②

→ Let say we need to execute instruction stored at 0000 0008. i.e after executing instruction stored at 0000 0004. i.e we need to change the order of execution. this can be done by using branching

→

Memory Add	Stored instruction
0000 0000	Instruction-1
0000 0004	" 2
" 0008	" 3
" 000C	" 4
⋮	⋮
0050	
0054	
0058	



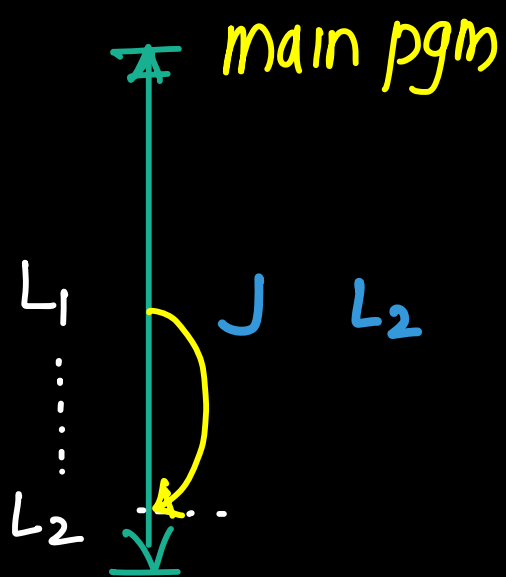
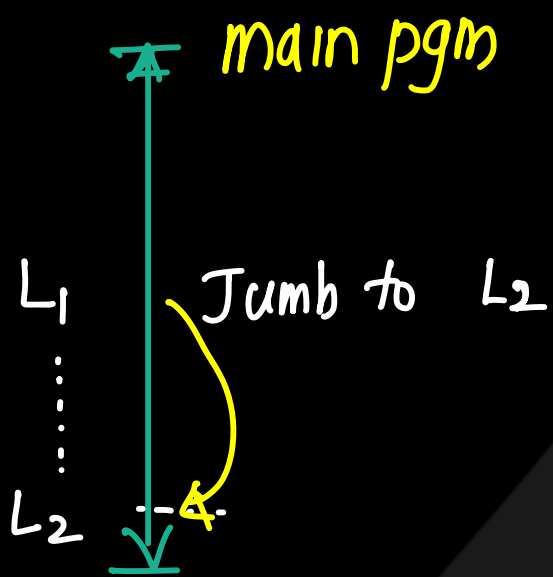
- Branching is the programming concept used to alter the normal sequential flow of execution of instructions.
- Branching can be done using Branch instruction.
- Branch instructions allow the processor to jump to instruction stored at different memory location.

Branch instructions are classified into two types. (2)

- ① Unconditional branch instruction
- ② Conditional branch instruction

① Unconditional branch instruction

Unconditional branch transfer control to another part of the program without checking any condition



When processor execute 'J L₂' instruction processor understand it a branch instruction, processor change the order of execution and processor execute instruction stored at memory location (L₂)

unconditional Branch instructions

- ① jump(j)
- ② jump and link(jal)

① jump(j)

→ it is a unconditional branch instruction

→ Format: j label

eg:

eg:

instruction 1
instruction 2
j target
instruction 3
instruction 4
instruction 5
instruction 6

target: instruction 7

When processor execute 'j target' instruction, then processor execute the instruction stored at memory location with address 'target'.

(2)

② jump and link (jal)

— this instruction perform two operation

① jump to the target address

② store the return address a register.

Format jal register, target Address

eg: jal x1, target

→ jump to the target address and return address stored in the x1 register

Conditional Jump instruction

(2)

→ the branch instruction transfer the control to the target address only if a specified condition is satisfied. If condition is false, execution continues with the next instruction.

→ Conditional jump instructions are

- | | |
|--|---|
| ① beq (Branch if equal) | = |
| ② bne (Branch if not equal) | ≠ |
| ③ blt (Branch if less than) | < |
| ④ bge (Branch if greater than or equal to) | ≥ |

① Branch if equal (beq)

Format beq register1, register2, target Address,

eg: beq \$0, \$1, target

If the content of \$0, \$1 are equal then branches to the target address.

② branch if not equal bne

Format bne register1, register2, target address

eg: bne \$0, \$1, target

Branches to target only if contents of \$0, \$1 are not equal.

③ branch if less-than (blt)

Format blt register1, register2, target Add

blt s0, s1, target

if content of s0 is less than s1, then branches to target Address

④ branch if greater than or equals (bge)

Format bge register1, register2, target Address

bge s0, s1, target.

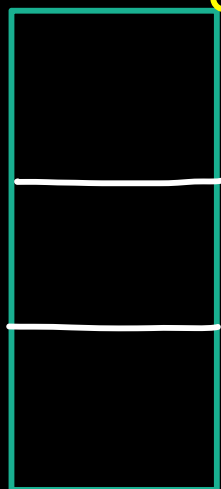


M SIGMA
GOKULAM

Stack. (Note)

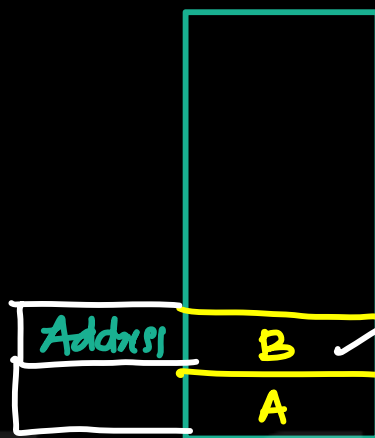
→ Stack is a special region of memory to store temporary data during the program execution.

memory



} stack

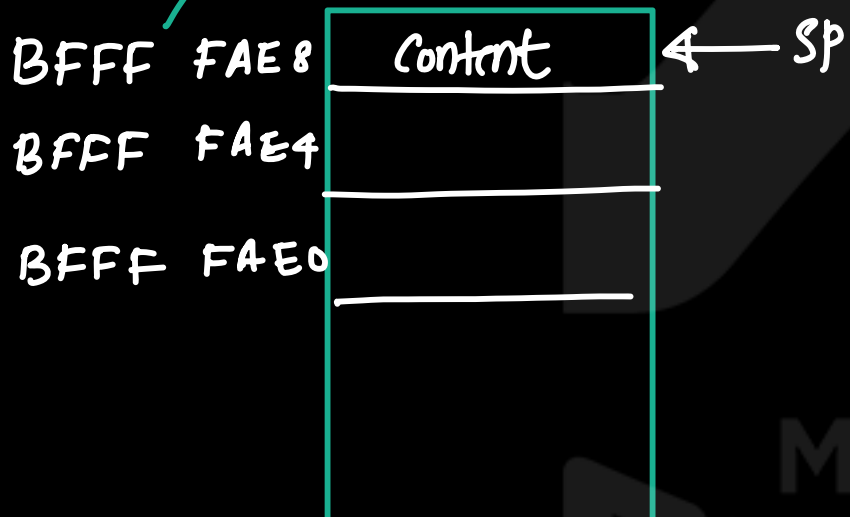
→ stack operate in last in first out principle



last in

→ Address.

Stack



— top of the stack is represented by a register called stack pointer. (SP).

→ Stack pointer is a register that contains address of the top of the stack.

SP BFFF FAE8

→ Basic operation in stack.

there are two operations are possible in stack

① push ② pop

① push

→ transfer data to stack

② POP → Retrieve the data from the stack

→ Functions of stack

① storing return address

when a function is called, the return address is stored in the stack so that control can return to the caller after execution

eg:

Return (ra) Address	addi s0, zero, zero
	jal target
	addi s0, s0, s0
target	add a0, a2, a1
	add s7, a0, 0
	jx ra

When target function called return address is stored in stack.

③ storing local variables

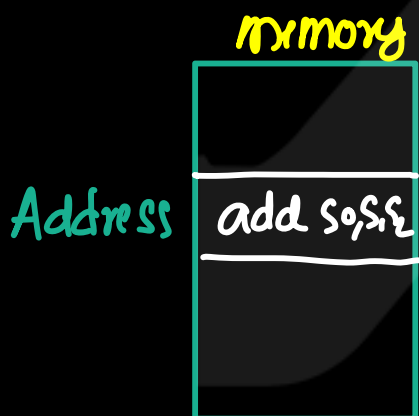
local variables declared inside a function then variable, are stored in stack but the variable exist only while the function is executing.

Instruction execution cycle (Note)

— CPU execute a task through the following steps.
① Fetch ② Decode ③ execute ④ store.

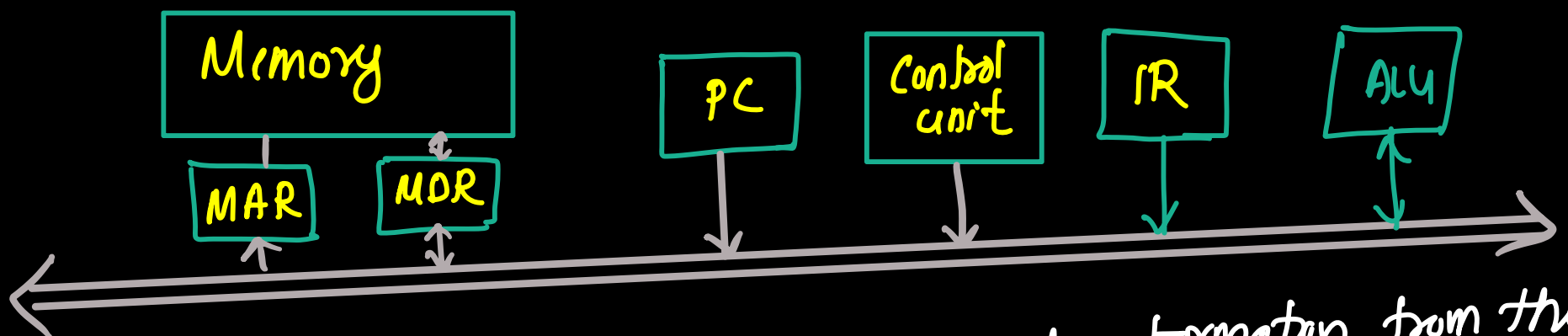
→ ① Fetch : Fetch the instruction from the memory
② decode : Control unit decode the instruction. understand the basic operation needs to perform
③ execute : execute the instruction

→ Consider an example of adding two numbers.
add s_0, s_1, s_2
this instruction stored at a memory location



① Program Counter : it is a register hold the address of next instruction to be executed.
(PC)

process
① Program Counter transfer the content to a register called memory address register



② Control unit send a signal to read information from the memory. instruction is fetched from the memory and placed in the Memory data register

- ③ Content of MDR is transferred to instruction register (IR) now fetch process is completed.
- ④ Control unit decodes the instruction stored in instruction register and determine the operation need to perform. Since we are considering an arithmetical instruction and it's done by ALU. so control unit send signal to ALU.
- ⑤ ALU execute the instruction and result is stored in register or memory location. thus execution of instruction is completed.

Summary:

The instruction fetch - execution cycle describes the sequence of operations performed by the CPU to fetch, decode, execute and complete one instruction.

Fetch

- PC holds the address of the next instruction.
- The instruction stored at the memory location pointed by PC is fetched.
- Fetched instruction is loaded into the IR.

decode

- Control unit decodes the instruction stored in IR.

execute

- CPU performs the operation specified by the instruction. store the result of execution in a register or memory if required.

Machine language (Note)

- Machine language is a low level language consists of binary instructions that are directly understood and executed by computer hardware
- **instructions** these are the commands given to computer hardware to perform a task. machine language instructions are represented using binary '1' and '0'
Eg: instruction in assembly language { instruction in machine language
add s0, s1, s2 1011101110110110
- In RISC architecture each instruction is represented by 32 bit

Instruction format

- ## Instruction format
- In machine language each instruction is represented by 32 bit binary numbers
 - Instruction Format specifies how these 32 bits are divided into different field.

— Different field in instruction

eg: add s_0, s_1, s_2
 ↓ ↘
opcode destination register
 source register

`addi` `s0, s1, 5`

↓
immediate
value

- ① Field for representing opcode, source register, destination register, immediate operands,
- ② Field for representing the basic operation

- Different instruction format (register)

- ① R type (Register) instructions
- ② I type (Immediate) instructions
- ③ S/B type (store/branch) instructions
- ④ J type (jump) instructions

① R type instruction Format. (Not)

— eg: add s_0, s_1, s_2

— these instructions uses 3 Register for representing operands .

— two source registers and one destination register

→ This instruction has 7 fields

① Field for representing opcode,

② Field for representing register.

③ Field for representing function

→ R type

funct7	s_1	s_2	funct3	rd	OP
7bit	5bit	5bit	3bit	5bit	7bit

OP → this field is represent the opcode. (7bit)

rd — this field is to represents destination register

s_1 & s_2 → this fields are used to represent source register (5bit each)

function 3, funct7 → these field is used to represent function

function 3, funct7 (function fields) and op field are used to represent the basic operation need to perform.

I-type instruction Format (Note)

— eg: `addi s0, s1, 5` immediate value.

`andi s0, s1, 5`

`ori s0, s1, 6`

`xori s0, s1, 5`

— Instruction format have the following fields

① OP (7bit) → Representing opcode

② rd (5bit) — Representing destination Register

③ funct3 (3bit) — function field.

④ rs1 (5bit) — Representing source register

⑤ imm (12bit) — Representing immediate value



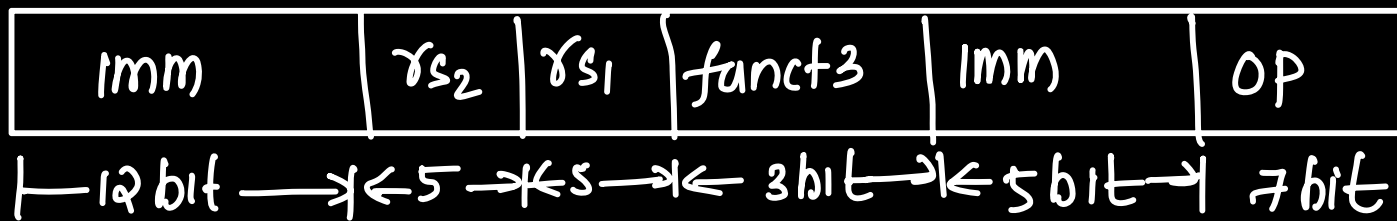
← 12bit → ← 5bit → ← 3bit → ← 5bit → ← 7bit →

imm field is used to represent the immediate value

M SIGMA
GOKULAM

B-type instruction format (Note)

- eg beq, s0, s2, target
- this instruction has two register operands and one immediate operand
- Replace funct7, rd field of R-type instruction with the 12 bit immediate imm



- imm field is used to represent immediate value.



(J-type-instructions) (Note)

— instruction format



← 20 bit → ← 5 bit → ← 7 bit →

- op field is used to represent opcode (7 bit)
- rd is used to represent the destination (5 bit)
- imm is used to represent immediate value (20 bit)



Addressing Modes (Note)

— Addressing mode specifies how operands are represented in an instruction

Different types of Addressing modes

- ① Register addressing modes
- ② Immediate addressing modes
- ③ Base addressing modes
- ④ PC Relative addressing modes.



1. Register Addressing mode (Note)

— In this addressing mode operands are present in the register inside the CPU. These registers are directly represented in the instruction

eg: $\text{add } s_0, s_1, s_2$

$$s_0 = s_1 + s_2$$

— operands are present in the registers s_1, s_2

— Result will be available in register s_0 .

② Immediate Addressing mode

— In this addressing mode one operand value is directly specified in the instruction

— eg: $\text{addi } s_0, s_1, 5$

$5 \rightarrow$ immediate value

$$s_0 = \text{content of } s_1 + 5$$

③ Base addressing mode

— memory access instructions such as load word (lw) and store word (sw), use base addressing

eg: $\text{lw } t_2, 3(s_2)$

operand is present on a memory location whose effective address is calculated by adding the base address stored in the register and the immediate value

$$\text{effective address} = \text{offset} + \text{base address stored in the register}$$

Here base address is stored in s_2

s_2 $0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0015$

offset = 3

effective Add = $0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0008$

Content of this memory location is copied to register t_2